

Perceptron simple y monocapa: representación de OR y XOR usando el bias estándar de PyTorch

(Trabajo en progreso...)

REDES NEURONALES 2026¹

¹Reportar errores a: tristan.osan@unc.edu.ar

Objetivo del tutorial

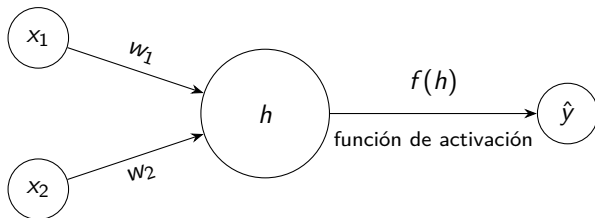
Pregunta guía

¿Por qué una neurona con frontera lineal puede aprender OR, pero no puede representar XOR?

El recorrido será:

- 1 definir OR y XOR como problemas de clasificación binaria;
- 2 describir una neurona con dos variables de entrada y un bias aditivo;
- 3 interpretar geométricamente la frontera de decisión;
- 4 estudiar el descenso por gradiente;
- 5 conectar las ecuaciones con una implementación en PyTorch;
- 6 mostrar por qué una capa oculta cambia la capacidad del modelo.

Nuestra primera red neuronal: el perceptron simple



La preactivación h es:

$$h = w_1x_1 + w_2x_2 + b.$$

La salida es:

$$\hat{y} = f(h).$$

En esta implementación, b es el **bias aditivo estándar de PyTorch**; no se incorpora como un tercer peso asociado a una entrada constante. Los parámetros w_1 , w_2 y b son coeficientes que **la neurona debe aprender** durante el *proceso de entrenamiento*.

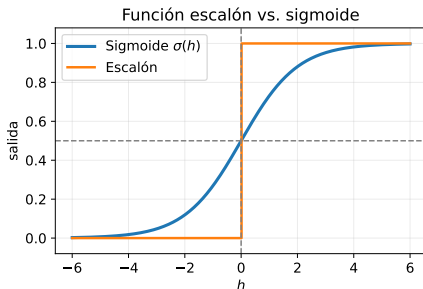
De la preactivación a una salida

La neurona calcula primero:

$$h = w_1x_1 + w_2x_2 + b.$$

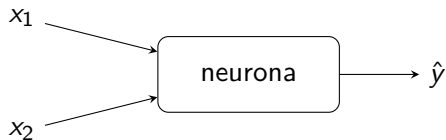
Luego aplica una función de activación. En la implementación en Google Colab vamos a usar como función de activación la *sigmoide*:

$$f(h) = \sigma(h) = \frac{1}{1 + e^{-h}}.$$



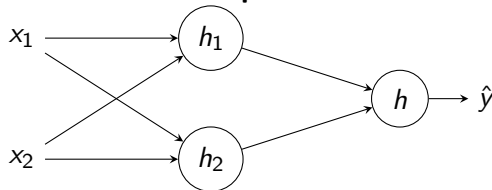
Arquitecturas que vamos a comparar

Sin capa oculta



$2 \rightarrow 1$ con bias aditivo.

Con capa oculta



$2 \rightarrow 2 \rightarrow 1$.

La primera arquitectura sirve para entender bias, frontera lineal, gradientes y separabilidad lineal. La segunda muestra cómo cambia la capacidad de representación.

Entradas binarias y salidas binarias

Trabajaremos con dos variables binarias:

$$x_1, x_2 \in \{0, 1\}.$$

Existen solamente cuatro entradas posibles:

$$(0, 0), \quad (0, 1), \quad (1, 0), \quad (1, 1).$$

Interpretación como clasificación

Cada compuerta lógica asigna una etiqueta $y \in \{0, 1\}$ a cada par (x_1, x_2) .

En este problema no estamos estudiando generalización a nuevos pares binarios: el dominio completo tiene cuatro elementos (cada elemento es un par de valores (x_1, x_2)). El objetivo es estudiar qué funciones puede representar una arquitectura.

Definición de OR

La compuerta OR vale 1 si al menos una entrada vale 1.

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Equivalentemente:

$$\text{OR}(x_1, x_2) = 1 \iff x_1 + x_2 \geq 1.$$

Definición de XOR

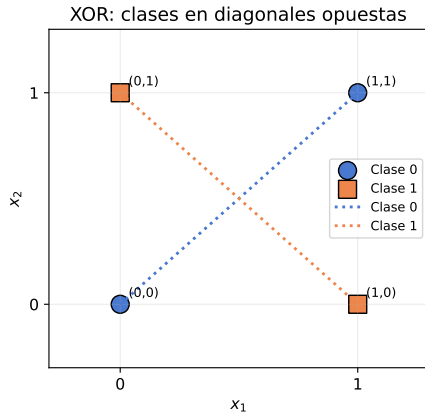
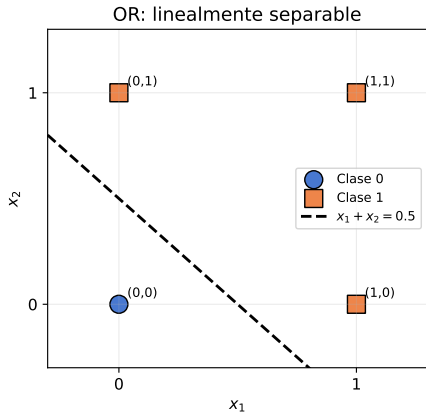
La compuerta XOR vale 1 cuando las entradas son distintas.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Equivalentemente:

$$\text{XOR}(x_1, x_2) = 1 \iff x_1 \neq x_2.$$

OR y XOR en el plano de entradas



En OR, las clases pueden separarse con una recta. En XOR, las clases quedan en diagonales opuestas.

La convención estándar de PyTorch

En esta implementación usamos directamente:

$$\mathbf{x} = (x_1, x_2).$$

La capa lineal de PyTorch calcula:

$$h = w_1x_1 + w_2x_2 + b.$$

Aquí:

- w_1 y w_2 son pesos multiplicativos;
- b es el bias aditivo de la neurona;
- b puede ser positivo, negativo o cero.

Frontera de decisión

La frontera se obtiene imponiendo $h = 0$:

$$w_1x_1 + w_2x_2 + b = 0.$$

Bias y umbral: relación de signos

A veces es útil hablar de un umbral θ que debe ser superado:

$$w_1x_1 + w_2x_2 \geq \theta.$$

Pero con la convención de PyTorch la condición de la clase 1 es:

$$w_1x_1 + w_2x_2 + b \geq 0.$$

Equivalentemente:

$$w_1x_1 + w_2x_2 \geq -b.$$

Por lo tanto:

$$\boxed{\theta = -b.}$$

Cuidado

El bias de PyTorch no es el umbral. El umbral es su opuesto.

Por qué usamos la sigmoide

El perceptrón clásico usa una función escalón:

$$\text{step}(h) = \begin{cases} 1, & h \geq 0, \\ 0, & h < 0. \end{cases}$$

Pero el escalón no es adecuado para utilizar el *método de descenso por gradiente*: no es derivable en 0 y su derivada es cero casi en todas partes.

La sigmoide, en cambio, es suave y derivable:

$$\sigma'(h) = \sigma(h)(1 - \sigma(h)).$$

Punto clave

La sigmoide suaviza la salida, pero no cambia el hecho de que la frontera de decisión sea lineal.

Umbral de decisión en la salida sigmoidea

Como

$$\sigma(0) = 0.5$$

y σ es creciente, entonces:

$$\hat{y} \geq 0.5 \iff h \geq 0.$$

Con la convención de PyTorch:

$$h = w_1x_1 + w_2x_2 + b.$$

Por lo tanto:

$$\hat{y} \geq 0.5 \iff w_1x_1 + w_2x_2 + b \geq 0.$$

Frontera de decisión

$$w_1x_1 + w_2x_2 + b = 0.$$

En dos dimensiones, esto es una recta.

Clasificación y frontera de decisión

Como

$$\sigma(0) = 0.5$$

y la función sigmoide es estrictamente creciente, el valor $\hat{y} = 0.5$ corresponde exactamente a $h = 0$.

Con la convención de PyTorch,

$$h = w_1x_1 + w_2x_2 + b.$$

Regla de clasificación

$$\hat{y} < 0.5 \iff h < 0 \iff w_1x_1 + w_2x_2 + b < 0$$

$$\hat{y} \geq 0.5 \iff h \geq 0 \iff w_1x_1 + w_2x_2 + b \geq 0$$

Por lo tanto:

$$\text{clase 0} \iff h < 0, \quad \text{clase 1} \iff h \geq 0.$$

Clasificación y frontera de decisión

Para la compuerta OR

El modelo debe aprender parámetros tales que:

$$(0,0) \longrightarrow h < 0 \longrightarrow \text{clase 0,}$$

mientras que

$$(0,1), (1,0), (1,1) \longrightarrow h \geq 0 \longrightarrow \text{clase 1.}$$

Frontera de decisión

La frontera corresponde al punto exacto en el que cambia la clasificación:

$$\hat{y} = 0.5 \iff h = 0.$$

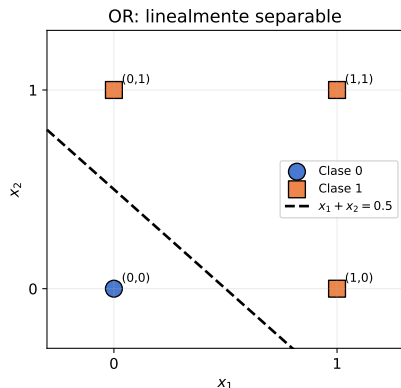
Por lo tanto,

$w_1x_1 + w_2x_2 + b = 0$. En dos dimensiones, esta ecuación representa una recta.

OR es linealmente separable

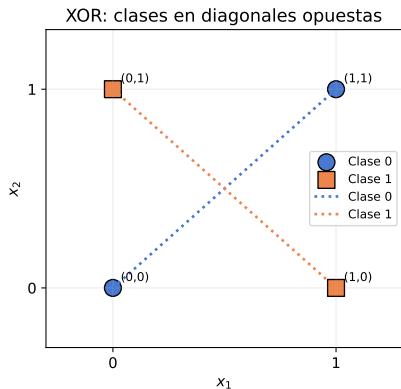
Para OR, **una recta posible** es:

$$x_1 + x_2 = 0.5.$$



- $(0,0)$ queda con $x_1 + x_2 = 0$.
- Los otros tres puntos quedan con $x_1 + x_2 \geq 1$.
- La arquitectura lineal tiene capacidad para representar OR.

XOR no es linealmente separable



En XOR:

- clase 0: (0, 0) y (1, 1);
- clase 1: (0, 1) y (1, 0).

Las clases ocupan diagonales opuestas del cuadrado. Una única recta no puede dejar ambas clases completamente separadas.

XOR no es linealmente separable: idea intuitiva

Usamos la convención:

$$h = w_1x_1 + w_2x_2 + b.$$

La neurona clasifica como clase 1 cuando:

$$h \geq 0 \iff w_1x_1 + w_2x_2 \geq -b.$$

Definimos el umbral:

$$\theta = -b.$$

Interpretación

La suma ponderada $w_1x_1 + w_2x_2$ debe superar el umbral θ .

Qué exige XOR

La tabla de XOR es:

x_1	x_2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Qué exige XOR

La tabla de XOR es:

x_1	x_2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Para que $(0, 0)$ sea clase 0, necesitamos:

$$b < 0.$$

Qué exige XOR

La tabla de XOR es:

x_1	x_2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Para que $(0, 0)$ sea clase 0, necesitamos:

$$b < 0.$$

Para que $(1, 0)$ sea clase 1, necesitamos:

$$w_1 + b \geq 0 \implies w_1 \geq -b.$$

Qué exige XOR

La tabla de XOR es:

x_1	x_2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Para que $(0, 0)$ sea clase 0, necesitamos:

$$b < 0.$$

Para que $(1, 0)$ sea clase 1, necesitamos:

$$w_1 + b \geq 0 \implies w_1 \geq -b.$$

Para que $(0, 1)$ sea clase 1, necesitamos:

$$w_2 + b \geq 0 \implies w_2 \geq -b.$$

La contradicción

Hasta ahora obtuvimos:

$$b < 0, \quad w_1 \geq -b, \quad w_2 \geq -b.$$

La contradicción

Hasta ahora obtuvimos:

$$b < 0, \quad w_1 \geq -b, \quad w_2 \geq -b.$$

Sumando las dos últimas desigualdades:

$$w_1 + w_2 \geq -2b.$$

La contradicción

Hasta ahora obtuvimos:

$$b < 0, \quad w_1 \geq -b, \quad w_2 \geq -b.$$

Sumando las dos últimas desigualdades:

$$w_1 + w_2 \geq -2b.$$

Como $b < 0$, entonces $-b > 0$. Por lo tanto:

$$-2b > -b.$$

Así:

$$w_1 + w_2 > -b.$$

La contradicción

Hasta ahora obtuvimos:

$$b < 0, \quad w_1 \geq -b, \quad w_2 \geq -b.$$

Sumando las dos últimas desigualdades:

$$w_1 + w_2 \geq -2b.$$

Como $b < 0$, entonces $-b > 0$. Por lo tanto:

$$-2b > -b.$$

Así:

$$w_1 + w_2 > -b.$$

Pero XOR exige que $(1, 1)$ sea clase 0:

$$w_1 + w_2 + b < 0 \quad \Longleftrightarrow \quad w_1 + w_2 < -b.$$

Contradicción

No puede ocurrir simultáneamente que

$$w_1 + w_2 > -b \quad \text{y} \quad w_1 + w_2 < -b.$$

Por lo tanto, no existen w_1 , w_2 y b que permitan a una sola neurona representar correctamente XOR.

Conclusión

No existen valores de w_1 , w_2 y b tales que una única neurona satisfaga las cuatro condiciones de XOR.

XOR no es linealmente separable.

Intuición

XOR le pide a la neurona:

x_1 solo $\Rightarrow$ activar,

x_2 solo $\Rightarrow$ activar,

pero

$x_1 + x_2 \Rightarrow$ no activar.

Una neurona que solo suma contribuciones ponderadas y compara contra un único umbral no puede cumplir esas tres condiciones a la vez.

Conclusión sobre la arquitectura sin capa oculta

Arquitectura

$$(x_1, x_2) \longrightarrow \text{una neurona} \longrightarrow \hat{y}.$$

Capacidad geométrica

Solo puede definir fronteras de la forma:

$$w_1x_1 + w_2x_2 + b = 0.$$

Es decir: una recta en el plano de entradas.

Mensaje central

El fracaso con XOR no se debe a PyTorch, ni a la tasa de aprendizaje, ni al número de épocas. La solución no existe dentro de esta familia de modelos.

¿Qué significa entrenar una red neuronal?

Entrenar significa realizar un procedimiento para que la red aprenda los valores de los parámetros de manera tal que las predicciones se parezcan a las salidas deseadas.

Los parámetros son:

$$w_1, \quad w_2, \quad b.$$

¿Qué significa entrenar una red neuronal?

Entrenar significa elegir los parámetros para que las predicciones se parezcan a las salidas deseadas.

Los parámetros son:

$$w_1, \quad w_2, \quad b.$$

Para cada ejemplo:

$$h^{(m)} = w_1 x_1^{(m)} + w_2 x_2^{(m)} + b, \quad \hat{y}^{(m)} = \sigma(h^{(m)}).$$

Necesitamos una función que mida el *error global del modelo*.

Función costo

En la notebook usamos el error cuadrático medio (MSE) con el factor 1/2:

$$E(w_1, w_2, b) = \frac{1}{2M} \sum_{m=1}^M \left(\hat{y}^{(m)} - y^{(m)} \right)^2.$$

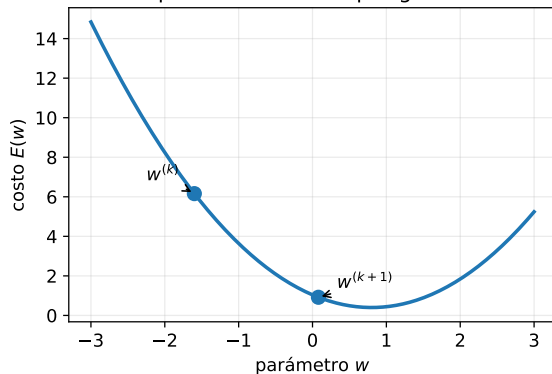
- M es el número de ejemplos.
- $y^{(m)}$ es la salida correcta.
- $\hat{y}^{(m)}$ es la predicción de la red.

Rol de E

Convierte el desempeño del modelo en un único número. Entrenar será intentar reducir ese número.

Intuición del descenso por gradiente

Un paso de descenso por gradiente



En una dimensión:

$$w^{(k+1)} = w^{(k)} - \eta \frac{dE}{dw}(w^{(k)}).$$

- Si la pendiente es positiva, reducimos w .
- Si la pendiente es negativa, aumentamos w .
- η controla el tamaño del paso.

Descenso por gradiente en varios parámetros

Ahora tenemos tres parámetros:

$$(w_1, w_2, b).$$

El gradiente es el vector de derivadas parciales:

$$\nabla E = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \frac{\partial E}{\partial b} \right).$$

La actualización es:

$$\begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix}^{(k+1)} = \begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix}^{(k)} - \eta \nabla E.$$

Idea

Mover los parámetros en la dirección opuesta al gradiente disminuye el costo localmente.

Regla de la cadena para un ejemplo

Para un ejemplo m :

$$E^{(m)} = \frac{1}{2}(\hat{y}^{(m)} - y^{(m)})^2,$$

$$\hat{y}^{(m)} = \sigma(h^{(m)}), \quad h^{(m)} = w_1 x_1^{(m)} + w_2 x_2^{(m)} + b.$$

Por la regla de la cadena:

$$\frac{\partial E^{(m)}}{\partial w_i} = \frac{\partial E^{(m)}}{\partial \hat{y}^{(m)}} \frac{\partial \hat{y}^{(m)}}{\partial h^{(m)}} \frac{\partial h^{(m)}}{\partial w_i}.$$

Los tres factores son:

$$\frac{\partial E^{(m)}}{\partial \hat{y}^{(m)}} = \hat{y}^{(m)} - y^{(m)},$$

$$\frac{\partial \hat{y}^{(m)}}{\partial h^{(m)}} = \hat{y}^{(m)}(1 - \hat{y}^{(m)}), \quad \frac{\partial h^{(m)}}{\partial w_i} = x_i^{(m)}.$$

Gradiente de los pesos y del bias

Para $i = 1, 2$:

$$\frac{\partial E^{(m)}}{\partial w_i} = (\hat{y}^{(m)} - y^{(m)})\hat{y}^{(m)}(1 - \hat{y}^{(m)})x_i^{(m)}.$$

Como:

$$\frac{\partial h^{(m)}}{\partial b} = 1,$$

para el bias obtenemos:

$$\frac{\partial E^{(m)}}{\partial b} = (\hat{y}^{(m)} - y^{(m)})\hat{y}^{(m)}(1 - \hat{y}^{(m)}).$$

Diferencia con la versión de entrada auxiliar

Aquí el bias se suma. Por eso su derivada entra con signo positivo. En la versión con entrada -1 , el parámetro asociado a esa entrada tenía derivada proporcional a -1 .

Optimización con descenso por gradiente

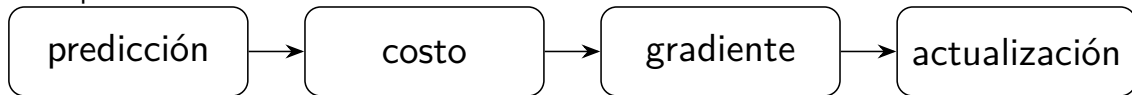
En la notebook usamos los cuatro ejemplos a la vez. Por eso:

$$E = \frac{1}{M} \sum_{m=1}^M E^{(m)}.$$

El gradiente total es el promedio de las contribuciones:

$$\nabla E = \frac{1}{M} \sum_{m=1}^M \nabla E^{(m)}.$$

Una época es:



Costo antes y después de actualizar

En cada época distinguimos:

$$E_{\text{antes}}^{(k)} = E(\theta^{(k)}),$$

que se usa para calcular el gradiente, y:

$$E_{\text{despues}}^{(k)} = E(\theta^{(k+1)}),$$

que mide cómo quedó el modelo luego de la actualización. Aquí θ representa el conjunto completo de parámetros.

Como el entrenamiento usa todo el conjunto de entrenamiento en cada actualización:

$$E_{\text{despues}}^{(k)} \approx E_{\text{antes}}^{(k+1)}.$$

Interpretación correcta

No son dos funciones costo distintas. Es la misma función evaluada en dos momentos de la época.

La neurona en PyTorch

La arquitectura sin capa oculta se implementa como:

```
modelo = nn.Sequential(  
    nn.Linear(2, 1),  
    nn.Sigmoid()  
)
```

La matriz de entrada es:

```
X = torch.tensor([  
    [0., 0.],  
    [0., 1.],  
    [1., 0.],  
    [1., 1.]  
)
```

Lectura

'Linear(2,1)' aprende dos pesos y un bias aditivo: w_1 , w_2 y b .

Dónde están los pesos y el bias

En PyTorch, los pesos de la primera capa se inspeccionan con:

```
modelo[0].weight
```

El bias se inspecciona por separado:

```
modelo[0].bias
```

Matemáticamente:

$$\text{modelo}[0].\text{weight} = (w_1, w_2), \quad \text{modelo}[0].\text{bias} = (b).$$

Forma de los tensores

Para 'Linear(2,1)', 'weight' tiene forma (1,2) y 'bias' tiene forma (1,). El bias es un vector con un valor por neurona de salida.

¿Qué significa entrenar una red neuronal?

Entrenar una red neuronal significa ajustar sus parámetros para que sus predicciones se acerquen a las salidas correctas.

En nuestro perceptrón simple, los parámetros aprendibles son:

$$w_1, \quad w_2, \quad b.$$

Para cada entrada (x_1, x_2) , el modelo calcula primero la preactivación:

$$h = w_1x_1 + w_2x_2 + b,$$

y luego aplica la función sigmoide:

$$\hat{y} = \sigma(h).$$

Objetivo del entrenamiento

Modificar w_1 , w_2 y b de manera que las predicciones $\hat{y}$ se aproximen a las salidas correctas y .

Ciclo básico de entrenamiento

En cada época de entrenamiento se repite el siguiente ciclo:

predicción $\longrightarrow$ costo $\longrightarrow$ gradientes $\longrightarrow$ actualización

- 1 El modelo calcula las predicciones $\hat{y}$.
- 2 Se calcula el costo E , que mide el error de las predicciones.
- 3 Se calculan los gradientes del costo respecto de los parámetros.
- 4 Los parámetros se actualizan en una dirección que busca reducir el costo.

Este proceso se repite durante muchas épocas.

1. Propagación hacia adelante

El primer paso consiste en calcular las predicciones del modelo.

Para cada ejemplo de entrenamiento:

$$h^{(m)} = w_1 x_1^{(m)} + w_2 x_2^{(m)} + b,$$

y luego:

$$\hat{y}^{(m)} = \sigma \left(h^{(m)} \right).$$

En PyTorch, esta operación se realiza simplemente mediante:

Forward pass

```
y_pred = modelo(X)
```

El modelo aplica sucesivamente todas las capas definidas en `nn.Sequential`.

2. Cálculo de la función de costo

La función de costo mide qué tan alejadas están las predicciones de las salidas correctas. En este ejemplo utilizamos:

$$E = \frac{1}{2M} \sum_{m=1}^M \left(\hat{y}^{(m)} - y^{(m)} \right)^2.$$

- Si E es grande, las predicciones están lejos de los valores correctos.
- Si E disminuye, el modelo está mejorando su ajuste.
- El objetivo del entrenamiento es encontrar parámetros que produzcan un costo pequeño.

3. Cálculo de los gradientes

El gradiente indica cómo cambia el costo cuando modificamos cada parámetro. En nuestro perceptrón queremos calcular:

$$\frac{\partial E}{\partial w_1}, \quad \frac{\partial E}{\partial w_2}, \quad \frac{\partial E}{\partial b}.$$

PyTorch calcula estos gradientes automáticamente mediante:

Backward pass

```
loss.backward()
```

Después de esta operación, los gradientes quedan almacenados en:

```
modelo[0].weight.grad
```

y

```
modelo[0].bias.grad.
```

¿Por qué debemos borrar los gradientes anteriores?

En PyTorch, los gradientes se **acumulan** por defecto.

Por lo tanto, antes de calcular los gradientes de una nueva época debemos eliminar los valores almacenados previamente.

Esto se realiza mediante:

Limpieza de gradientes

```
optimizador.zero_grad()
```

o, si no estamos utilizando todavía un optimizador:

Alternativa

```
modelo.zero_grad()
```

De esta forma, cada llamada a `backward()` calcula los gradientes correspondientes a la iteración actual.

4. Actualización de los parámetros

Una vez calculados los gradientes, los parámetros se actualizan mediante descenso por gradiente:

$$\theta_i^{[k+1]} = \theta_i^{[k]} - \eta \frac{\partial E}{\partial \theta_i},$$

donde:

$$\theta_i \in \{w_1, w_2, b\},$$

y η es la **tasa de aprendizaje**. Por ejemplo:

$$w_1^{[k+1]} = w_1^{[k]} - \eta \frac{\partial E}{\partial w_1}.$$

Idea central

El gradiente indica la dirección de mayor aumento del costo. Por eso nos desplazamos en la dirección opuesta.

El papel del optimizador

En lugar de actualizar manualmente cada parámetro, PyTorch puede hacerlo mediante un optimizador. Por ejemplo:

```
torch.optim.SGD
```

implementa descenso por gradiente estocástico. El optimizador se crea mediante:

Definición del optimizador

```
optimizador = torch.optim.SGD(  
    modelo.parameters(), lr=eta)
```

Luego, la actualización de los parámetros se realiza mediante:

Actualización

```
optimizador.step()
```

Una época de entrenamiento

Una época completa puede interpretarse como la secuencia:

$$\theta^{[k]} \longrightarrow \hat{y} \longrightarrow E(\theta^{[k]}) \longrightarrow \nabla E \longrightarrow \theta^{[k+1]}.$$

Es decir:

- 1 usamos los parámetros actuales;
- 2 calculamos las predicciones;
- 3 calculamos el costo;
- 4 calculamos los gradientes;
- 5 actualizamos los parámetros.

Después comienza una nueva época utilizando los parámetros actualizados.

Costo antes y después de la actualización

En cada época podemos distinguir dos valores del costo. Antes de actualizar:

$$E_{\text{antes}}^{[k]} = E \left(\boldsymbol{\theta}^{[k]} \right).$$

Después de actualizar:

$$E_{\text{después}}^{[k]} = E \left(\boldsymbol{\theta}^{[k+1]} \right).$$

La actualización es:

$$\boldsymbol{\theta}^{[k+1]} = \boldsymbol{\theta}^{[k]} - \eta \nabla E \left(\boldsymbol{\theta}^{[k]} \right).$$

Objetivo

Esperamos que, en general,

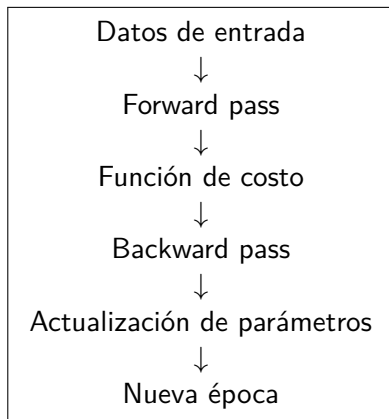
$$E_{\text{después}}^{[k]} < E_{\text{antes}}^{[k]}.$$

Entrenamiento en PyTorch

El ciclo de entrenamiento puede escribirse como:

```
for _ in range(epocas):  
    # 1. Forward pass  
    y_pred = modelo(X)  
    # 2. Costo  
    loss = 0.5 * criterio(y_pred, y)  
    # 3. Borrar gradientes anteriores  
    optimizador.zero_grad()  
    # 4. Calcular nuevos gradientes  
    loss.backward()  
    # 5. Actualizar parametros  
    optimizador.step()
```

Resumen del proceso de entrenamiento



- El proceso se repite hasta que el costo se estabiliza o se alcanza el número de épocas establecido.

Ciclo de entrenamiento en PyTorch

```
for epoca in range(epocas):  
    y_pred = modelo(X)  
    loss_antes = 0.5 * criterio(y_pred, y)  
  
    optimizador.zero_grad()  
    loss_antes.backward()  
    optimizador.step()  
  
    with torch.no_grad():  
        loss_despues = 0.5 * criterio(modelo(X), y)
```

- 'zero_grad()' borra gradientes acumulados.
- 'backward()' calcula gradientes.
- 'step()' actualiza parámetros.
- 'torch.no_grad()' evita construir un grafo para la evaluación posterior.

Qué calcula 'loss.backward()'

'loss.backward()' no modifica los pesos. Solo calcula y guarda:

$$\frac{\partial E}{\partial w_1}, \quad \frac{\partial E}{\partial w_2}, \quad \frac{\partial E}{\partial b}.$$

En PyTorch esos valores quedan separados:

```
modelo[0].weight.grad
```

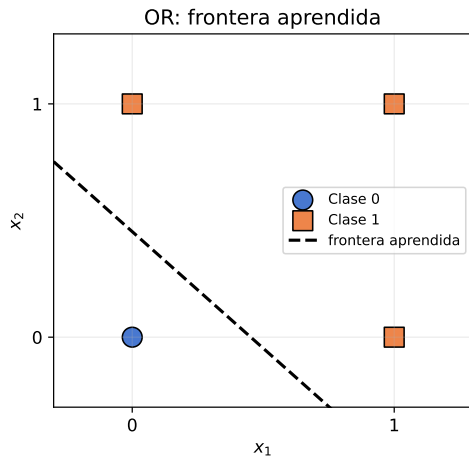
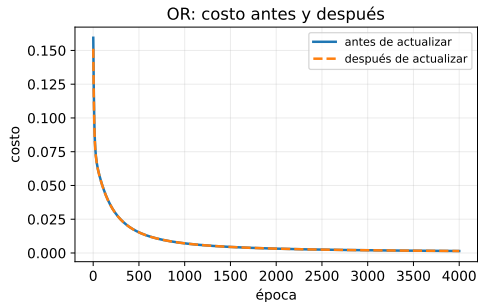
y

```
modelo[0].bias.grad.
```

Conexión con la teoría

PyTorch aplica automáticamente la regla de la cadena que derivamos manualmente.

Experimento 1: OR con una neurona



OR: costo decreciente y frontera lineal aprendida.

Interpretación del experimento OR

- El costo disminuye durante el entrenamiento.
- La frontera aprendida tiene la forma:

$$w_1x_1 + w_2x_2 + b = 0.$$

- Se obtienen 4/4 clasificaciones correctas.

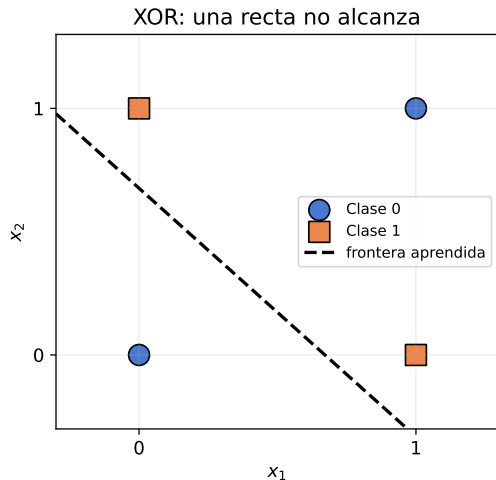
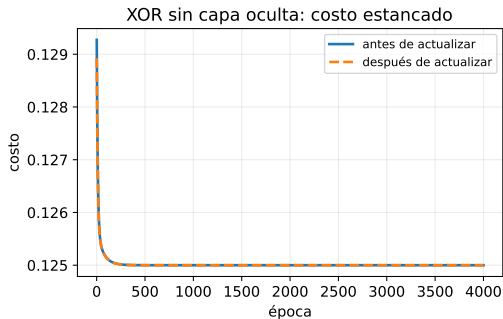
Conclusión

La arquitectura sin capa oculta tiene capacidad suficiente para representar OR.

Importante

No buscamos recuperar una recta única. Existen muchas rectas que separan OR correctamente.

Experimento 2: XOR con la misma neurona



El costo se estanca alrededor de 0.125 y no aparece una frontera lineal que separe las clases.

Qué no debemos concluir de XOR

Conclusiones incorrectas

- “PyTorch no pudo aprender XOR”.
- “Faltan más épocas”.
- “Hay pocos datos”.
- “La tasa de aprendizaje necesariamente está mal”.

Conclusión correcta

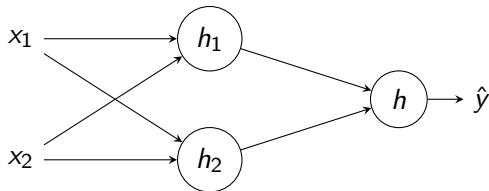
La arquitectura sin capa oculta solo puede representar una frontera lineal. XOR no pertenece a esa familia de funciones.

Si las salidas quedan cerca de 0.5, el conteo de aciertos con umbral ' ≥ 0.5 ' puede ser engañoso. Lo relevante es la geometría y el costo estancado.

Nueva arquitectura: $2 \rightarrow 2 \rightarrow 1$

Agregamos dos neuronas ocultas:

$$(x_1, x_2) \longrightarrow (\sigma(h_1), \sigma(h_2)) \longrightarrow \hat{y}.$$



La capa oculta permite combinar varias transformaciones lineales mediante una no linealidad.

Código PyTorch para la red multicapa

```
modelo_mlp = nn.Sequential(  
    nn.Linear(2, 2),  
    nn.Sigmoid(),  
    nn.Linear(2, 1),  
    nn.Sigmoid()  
)
```

Cada 'nn.Linear' usa la convención habitual de PyTorch:

$$h = \sum_i w_i x_i + b.$$

Activación elemento a elemento

La instrucción 'nn.Sigmoid()' aplica la sigmoide a cada componente del tensor que recibe. En la capa oculta transforma (h_1, h_2) en $(\sigma(h_1), \sigma(h_2))$.

Por qué la capa oculta ayuda

Cada neurona oculta calcula:

$$z_j = \sigma(w_{j1}x_1 + w_{j2}x_2 + b_j), \quad j = 1, 2.$$

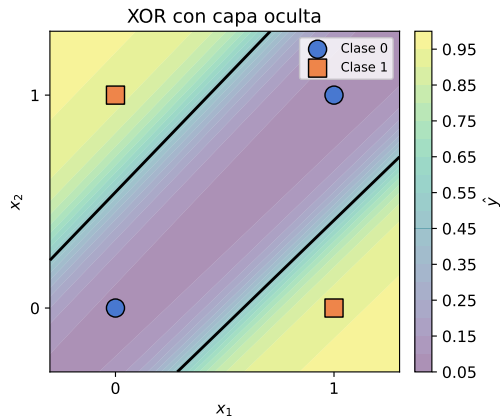
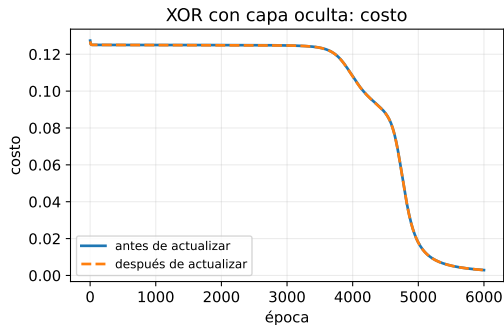
La neurona de salida combina esas representaciones:

$$\hat{y} = \sigma(v_1z_1 + v_2z_2 + c).$$

Cambio de capacidad

La salida ya no depende de (x_1, x_2) mediante una única combinación lineal. La red puede construir regiones de decisión no lineales en el plano original.

Experimento 3: XOR con capa oculta



Con una capa oculta, la red alcanza 4/4 clasificaciones correctas sobre XOR.

Comparación final

Arquitectura	Frontera	OR	XOR
Una neurona sin capa oculta	una recta	sí	no
$2 \rightarrow 2 \rightarrow 1$	no lineal en el plano original	sí	sí

Síntesis

Entrenar un modelo no garantiza que exista una solución dentro de la arquitectura elegida.

Ideas que deben quedar claras

- 1 OR y XOR usan las mismas cuatro entradas; cambian las etiquetas.
- 2 'nn.Linear(2,1)' implementa $h = w_1x_1 + w_2x_2 + b$.
- 3 Una neurona sin capa oculta define una frontera lineal.
- 4 OR es linealmente separable; XOR no lo es.
- 5 El descenso por gradiente actualiza pesos y bias usando derivadas del costo.
- 6 PyTorch calcula esas derivadas con autograd.
- 7 Una capa oculta aumenta la capacidad de representación.

Al abrir la notebook de Colab, conviene identificar explícitamente:

- dónde se define la matriz de entradas X con dos columnas;
- dónde aparece 'Linear(2,1)' con bias por defecto;
- dónde están 'modelo[0].weight' y 'modelo[0].bias';
- cuándo se calcula el costo antes de actualizar;
- cuándo se ejecuta 'backward()';
- cuándo se actualizan los parámetros con 'step()';
- cómo cambia la arquitectura al pasar a $2 \rightarrow 2 \rightarrow 1$.